Live SWOT Frontend Plan

Team: Frontend (2 people)

Stack: Next.js (App Router) + TypeScript on Vercel; FastAPI backend supplies research-run data

Mockup: Reference only (layout/IA), not a pixel spec. Screens follow planning.md.

Overview

Build a Next.js frontend for Live SWOT that implements the screens from planning.md, consumes FastAPI research-run data via a typed API client, and splits cleanly across two frontend owners so UI work can proceed with mocks before the backend is ready.

The UI never scrapes or invents SWOT content. All findings come from backend research runs / the database.

Core rule: every SWOT/finding point must show a source URL (and preferably a detected-at date).

Screens

Screen	Planning story	UI shows (from API)
Competitors list	Add competitor + see what changed	Tracked competitors, last research status/date, recent-change summary
Competitor profile	View profile + compare signals	Summary, SWOT findings, scores, source links, change markers, charts
SWOT overview	View overall SWOT	Aggregated S/W/O/T; each item links to competitor + source
Our Company	GitHub connect + baseline	Repo/file, sync status, SCADABLE feature baseline
Agent runs	View agent runs	Run list/detail, status, cost, cache hits/misses, competitor/PR link

Architecture (high level)

Users (Founders / Sales) -> Next.js Frontend (Vercel) -> FastAPI Backend -> Postgres

Research Workflow (Python) writes SWOT/findings/runs into Postgres after pulling external sources, GitHub product features, and LLM analysis. The frontend only reads/writes through the API.

Stack decisions

- Next.js App Router + TypeScript on Vercel
- CSS variables + CSS Modules ported from the mockup tokens
- TanStack Query for list/detail/run fetching, loading/error states, light polling
- Recharts for competitor-profile charts
- Typed API client in lib/api/ - never scatter fetch in components
- Mock-first mode: NEXT_PUBLIC_USE_MOCKS=true; flip env to hit live backend
- Auth MVP: Vercel Deployment Protection + shared API key header
- Repo layout: frontend/ at repo root. Env: NEXT_PUBLIC_API_URL

Folder structure

```
frontend/
  app/
    layout.tsx
    page.tsx                 # redirect -> /competitors
    competitors/page.tsx
    competitors/[id]/page.tsx
    overview/page.tsx
    company/page.tsx
    runs/page.tsx
    runs/[id]/page.tsx
  components/
    ui/                      # shared
    layout/                  # AppShell, Sidebar, Topbar
```

```
    competitors/              # FE A
    overview/                 # FE B
    company/                  # FE B
    runs/                     # FE A
  lib/api/ + mocks/
  styles/tokens.css
```

API resources

- GET/POST /competitors
- GET /competitors/:id (profile + findings + sources + changes + chart series)
- GET /swot/overview
- GET /company (+ GitHub connection status)
- GET /runs, GET /runs/:id
- POST /competitors returns competitor + queued run id

Profile chart series (from API; FE only renders)

- Category scores - 0-10 competitor vs SCADABLE (radar or grouped bar)
- Activity over time - findings/changes per week (line or area)
- Signals by source - website / jobs / reviews / press (donut or stacked bar)
- Hiring trend - open-role count over time when job data exists
- Empty series => empty chart state; do not invent numbers on the client

Screen build order (MVP)

- 1. App shell - nav routes for all five surfaces
- 2. Competitors list + Add competitor
- 3. Competitor profile - findings, sources, change badges, Recharts graphs
- 4. SWOT overview - four quadrants; click-through to profile/finding
- 5. Agent runs - table + expandable/detail steps
- 6. Our Company - baseline + GitHub sync status (connect stubbed until backend ready)

Two-person ownership

Fairness: FE A owns more product UI (list, profile, charts, runs). FE B owns Overview + Company UI plus deploy/ops (Vercel) and shared UI foundation (AppShell + components/ui). That keeps the split balanced.

Shared kickoff (half day, both): scaffold Next.js app, tokens, AppShell, lib/api types + mock client. FE B creates/owns the Vercel project from day one.

Owner	Owns	Do not touch without sync
FE A	competitors/*, Add modal, profile + charts, runs/*	overview/company; Vercel project settings
FE B	overview/*, company/*, shared UI + AppShell, Vercel	competitors/runs folders

FE A tasks

- Build /competitors list: search, filters, recent-change snippet, empty/loading/error
- Build Add Competitor modal: name + website validation, POST, toast + pending row
- Build /competitors/[id] profile: summary, findings with source links, change badges
- Add Recharts graphs: scores vs SCADABLE, activity over time, signals by source
- Build /runs list + detail: status, cost, steps, cache hits/misses, competitor link
- Add light polling on active runs via TanStack Query
- Own lib/api/competitors.ts, lib/api/runs.ts + mocks; keep DTOs aligned with FE B
- Wire list -> profile navigation and pending/failed/success status

FE B tasks

- Build /overview SWOT page: four quadrants; each item links to competitor + source
- Build /company page: SCADABLE feature baseline, GitHub repo/file/sync status
- Own shared AppShell + components/ui polish (Button, Modal, Table, Pill, Toast)
- Own lib/api/swot.ts, lib/api/company.ts + matching mock fixtures
- Own Vercel: project, repo link, prod + preview, env vars, Deployment Protection, README
- Keep main deploy green when FE A merges; check preview URLs on PRs

Integration seams: both only import from lib/api/* and components/ui/*. PRs stay vertical by route. Agree on DTO shapes in one PR before parallel UI work. FE B owns deploy breakage; FE A owns competitors/runs UI bugs.

Backend connectability

- Pages call hooks like useCompetitors() -> api.competitors.list(); mocks flip via env
- CORS: backend must allow the Vercel origin
- Empty/pending/error states are first-class
- No LLM prompts, scrape logic, or DB access in Next.js - that stays in Python

Out of scope for frontend

- Ingestion, change detection, LLM prompts, schedulers, Postgres schema
- GitHub App installation backend (FE only shows status / starts connect URL)
- Treating mockup sample companies as the source of truth

Success criteria

- Deployed on Vercel with all five routes usable against mocks
- Same build works against FastAPI by setting NEXT_PUBLIC_API_URL and turning mocks off
- Every finding in Overview + Profile shows a source link
- Competitor profile shows at least three charts driven by API/mock series
- Two owners can merge without constant conflicts via folder ownership
- Add competitor -> pending research -> profile fills when run completes